Assignment #2

AI-Based Mini Project in .NET

(Practical Lab Report)

EmotionApp:
AI-Based Sentiment Analysis Application

Submitted By:	Sareen Fatima
Roll Number:	232201061
Course:	AI-Based Application Development
Semester:	6th Semester
Department:	Computer Science
Institution:	KICSIT, IST Campus Kahuta

Submitted To:
Sir Uzair Hassan

Submission Date:
March 31, 2026

This report documents the development of an AI-powered text sentiment analysis application using .NET 8.0 and ML.NET.

Contents

1 Project Introduction **2**
- 1.1 Problem Statement . . . 2
- 1.2 Objective . . . 2

2 System Design **2**
- 2.1 Architecture Overview . . . 2
- 2.2 Workflow & Pipeline . . . 3
- 2.3 Model Selection: SdcaMaximumEntropy . . . 3
  - 2.3.1 Why This Model Was Chosen . . . 3
  - 2.3.2 How It Operates . . . 3

3 Tools & Technologies **4**
- 3.1 Development Environment . . . 4
- 3.2 Libraries and APIs . . . 4

4 Code Snippets **4**
- 4.1 ModelTrainer.cs - Pipeline Processing . . . 4
- 4.2 Program.cs - Environment Divergence . . . 4

5 Application Preview **5**

6 Testing **6**

7 Results & Discussion **7**
- 7.1 What Worked Well . . . 7
- 7.2 Limitations . . . 7

8 Future Improvements **7**

9 Docker Implementation **7**
- 9.1 Dockerfile . . . 7
- 9.2 Base Image Explanation . . . 8
- 9.3 Workflow (Multi-Stage Build) . . . 8
- 9.4 Run Commands . . . 8

10 Challenges & Solutions **9**
- 10.1 GUI Crashes in Headless Docker Environments . . . 9
- 10.2 Dataset Delimiter Conflicts . . . 9
- 10.3 Docker Build Context and File Pathing . . . 9

11 Conclusion **9**

References **10**

1 Project Introduction

1.1 Problem Statement

In today's information age, individuals and organizations are constantly faced with an overwhelming amount of text-based content—customer feedback, social media posts, support tickets, and chat logs. Reading and manually identifying the core emotional sentiment behind lengthy texts is time-consuming and inefficient at scale. Traditional keyword spotting often misses broader emotional context.

There is a critical need for a tool that can quickly parse sentences and predict the targeted human emotion, enabling rapid, objective context grouping without manual reading.

1.2 Objective

The objective of this project is to develop a user-friendly, locally trained AI-powered sentiment analysis application called **EmotionApp** that:

1. Accepts raw user text inputs.
2. Leverages local machine learning pipelines (ML.NET) to generate intelligent, context-aware emotion classifications (Joy, Anger, Sadness, etc.).
3. Provides a clean, intuitive Windows Forms interface for easy interaction on local desktop environments.
4. Implements a headless runtime detection trigger allowing execution inside isolated Docker containers without visual crashes.
5. Demonstrates the complete lifecycle of AI application development and training in .NET.

2 System Design

2.1 Architecture Overview

The EmotionApp application follows a segmented architecture with distinct operations. Below is a visual representation of the Multi-Tier Application Architecture:

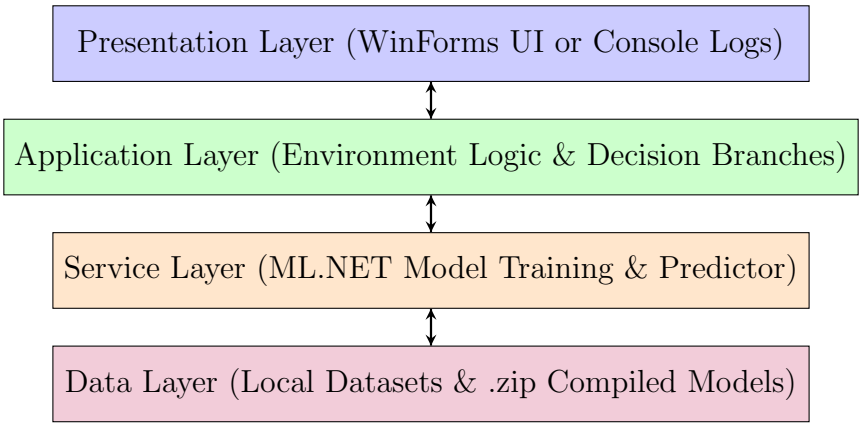

Figure 1: Multi-Tier System Architecture Diagram

2.2 Workflow & Pipeline

The pipeline training follows this sequential flow represented below:

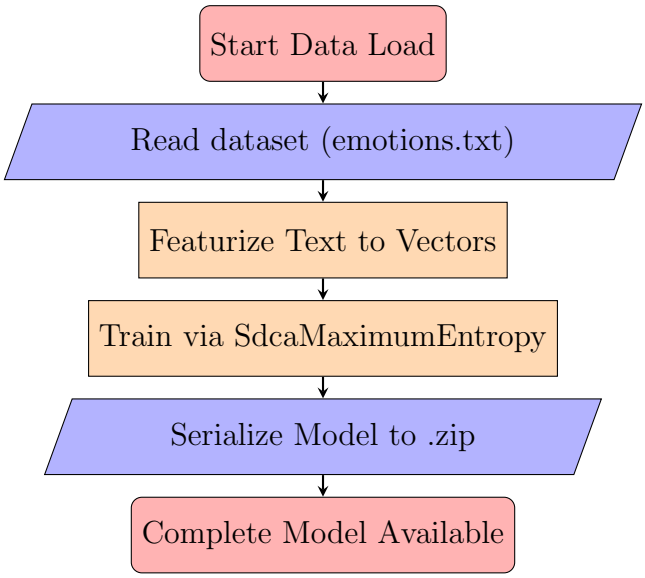

Figure 2: Workflow of the ML.NET Training Pipeline

2.3 Model Selection: SdcaMaximumEntropy

For the core predictive engine of EmotionApp, the **Stochastic Dual Coordinate Ascent (SDCA) Maximum Entropy** multiclass classification model was selected.

2.3.1 Why This Model Was Chosen

1. **Multiclass Suitability**: The system requires mapping user inputs into multiple distinct emotion buckets (Joy, Sadness, Anger, Fear, Love, Surprise). SDCA Maximum Entropy handles multi-bucket targets natively rather than executing basic binary (true/false) calculations.

2. **Local Resource Efficiency**: SDCA is highly optimized for performance in memory-restricted environments. It iterates rapidly over datasets to find an optimized point without needing massive GPU resources.

3. **High Text Tolerance**: Paired with ML.NET's featurizer, Maximum Entropy effectively weights conversational keywords to generate precise probability distributions.

2.3.2 How It Operates

The training pipeline featurizes the raw English text by breaking it into numerical vectors (N-grams). The Maximum Entropy algorithm then analyzes the frequency and structure of these vectors associated with labeled emotions. It attempts to find the probability distribution that best matches the characteristics of the training data. For any new, unseen string of text fed into the predictor, the model calculates a probability score array across all trained classes and ultimately outputs the label yielding the highest mathematical probability.

3 Tools & Technologies

3.1 Development Environment

Component	Technology
Framework	.NET 8.0
UI Technology	Windows Forms (Local Mode)
Programming Language	C#
IDE	Visual Studio 2022
Containerization	Docker Desktop

Table 1: Development Environment

3.2 Libraries and APIs

Library/API	Purpose	Version
Microsoft.ML	Complete AI pipeline and training	4.0.1 / 5.0.0
Windows Forms	Native user interface framework	Built-in

Table 2: Libraries Used

4 Code Snippets

4.1 ModelTrainer.cs - Pipeline Processing

This snippet handles compiling the pipeline and fitting data via Stochastic Dual Coordinate Ascent.

```
var pipeline = mlContext.Transforms.Conversion.MapValueToKey("Label", nameof(EmotionData.Emotion))
    .Append(mlContext.Transforms.Text.FeaturizeText("Features", nameof(EmotionData.Text)))
    .Append(mlContext.MulticlassClassification.Trainers.SdcaMaximumEntropy("Label", "Features"))
    .Append(mlContext.Transforms.Conversion.MapKeyToValue("PredictedLabel"));

var model = pipeline.Fit(trainingData);
```

Listing 1: ML.NET Model Training Workflow

4.2 Program.cs - Environment Divergence

This file triggers background console prediction behaviors whenever localized monitors are not detected.

```
bool isDocker = Environment.GetEnvironmentVariable("DOTNET_RUNNING_IN_CONTAINER") == "true";

if (isDocker)
```

```
{
    RunConsoleMode();
}
else
{
    RunUiMode();
}
```

Listing 2: Handling Docker vs Local Native UI Launch

5 Application Preview

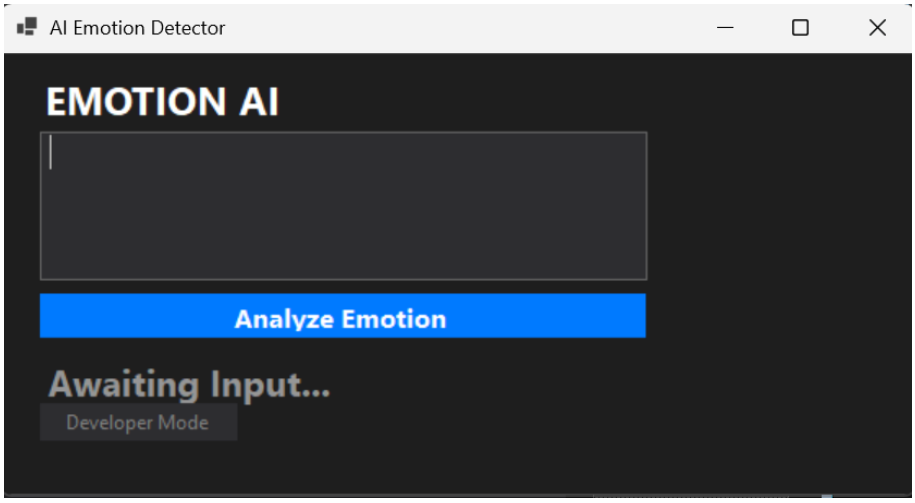

Figure 3: Application Standard Visual UI (Base State)

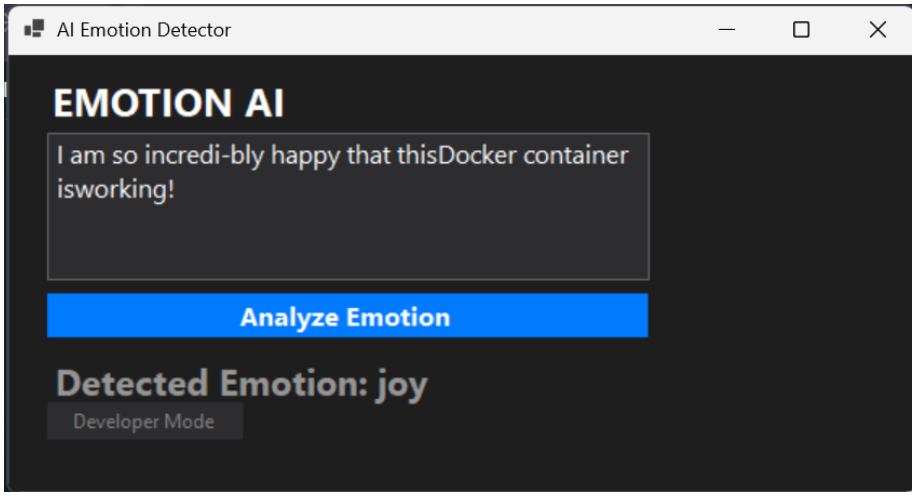

Figure 4: User UI Displaying Successfully Classified Emotion Label

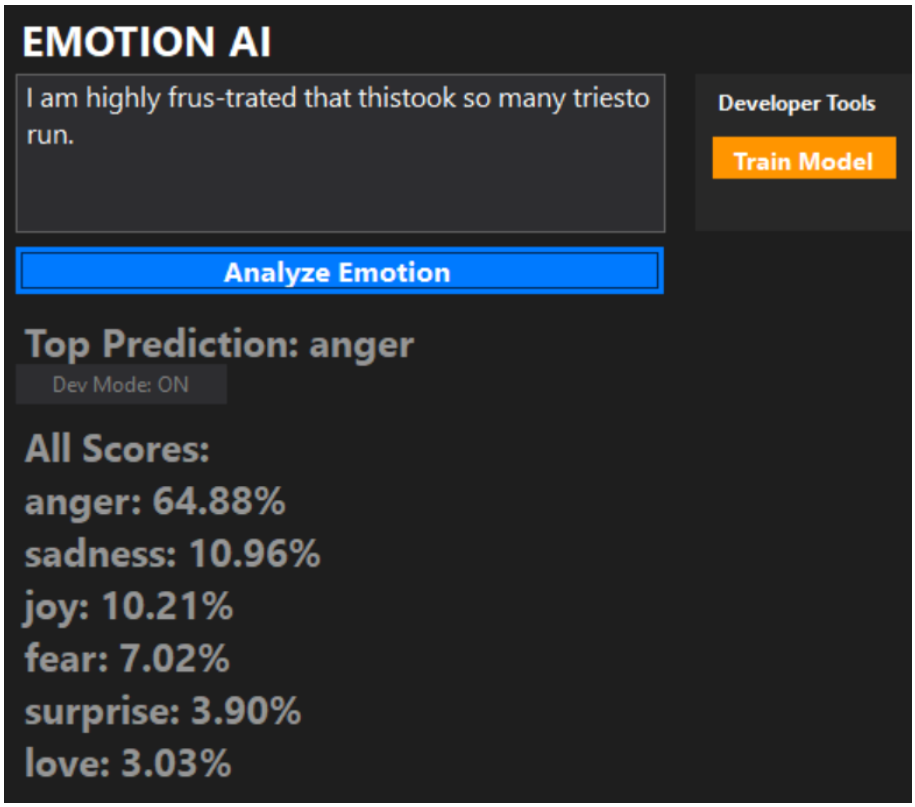

Figure 5: Developer Mode Execution within Visual Studio 2022

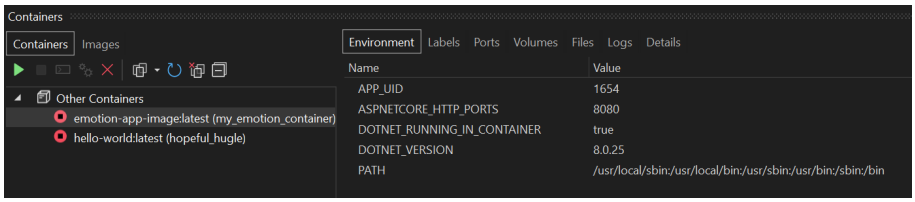

Figure 6: Application Successful Image Assembly Operation in Docker

6 Testing

Test Input	Expected Output	Result
I am so incredibly happy that this Docker container is working!	joy	Pass
I am highly frustrated that this took so many tries to run.	anger	Pass

Table 3: Test Cases and Results

7 Results & Discussion

7.1 What Worked Well

1. **Successful Model Assembly**: Local training operations fired off quickly and maintained localized security with zero public API pings.

2. **Target Separations**: Dividing project compilation into a specific `EmotionApp.Docker.csproj` kept development from bricking due to missing Linux visual framework libraries.

7.2 Limitations

1. **Context Bound**: Sentiment labels rely heavily on specific keyword scoring. Context structures heavily outside typical training data patterns may result in wrong labels.

2. **Windows Bound GUI**: Without utilizing containerization workarounds, cross-platform distribution of full native WinForms UIs is severely limited.

8 Future Improvements

1. **Web API Decoupling**: Transition from Windows Forms to a Web API setup so multiple container clients can query text predictions via endpoints.

2. **Hyperparameter Tuning**: Implementing dynamic parameters for the `SdcaMaximumEntropy` instance to achieve better precision on larger datasets.

9 Docker Implementation

9.1 Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

COPY ["EmotionApp.Docker.csproj", "./"]
RUN dotnet restore "EmotionApp.Docker.csproj"

COPY . .
RUN dotnet publish "EmotionApp.Docker.csproj" -c Release -o /app/
    publish

FROM mcr.microsoft.com/dotnet/runtime:8.0 AS final
WORKDIR /app
COPY --from=build /app/publish .

ENTRYPOINT ["dotnet", "EmotionApp.dll"]
```

Listing 3: Dockerfile for EmotionApp Containerization

9.2 Base Image Explanation

Base Image	Purpose
`mcr.microsoft.com/dotnet/sdk:8.0`	**Build Stage Image**: Includes standard libraries and compilers necessary to execute `dotnet restore` and `dotnet publish` on codebases.
`mcr.microsoft.com/dotnet/runtime:8.0`	**Runtime Stage Image**: A lightweight package containing strictly the runtime engine to execute completed binary files, keeping the image small.

Table 4: Docker Base Images and Their Purposes

9.3 Workflow (Multi-Stage Build)

To understand the isolation process handled in the Dockerfile above, consider the structural transfer from standard development kits to standard execution runtime:

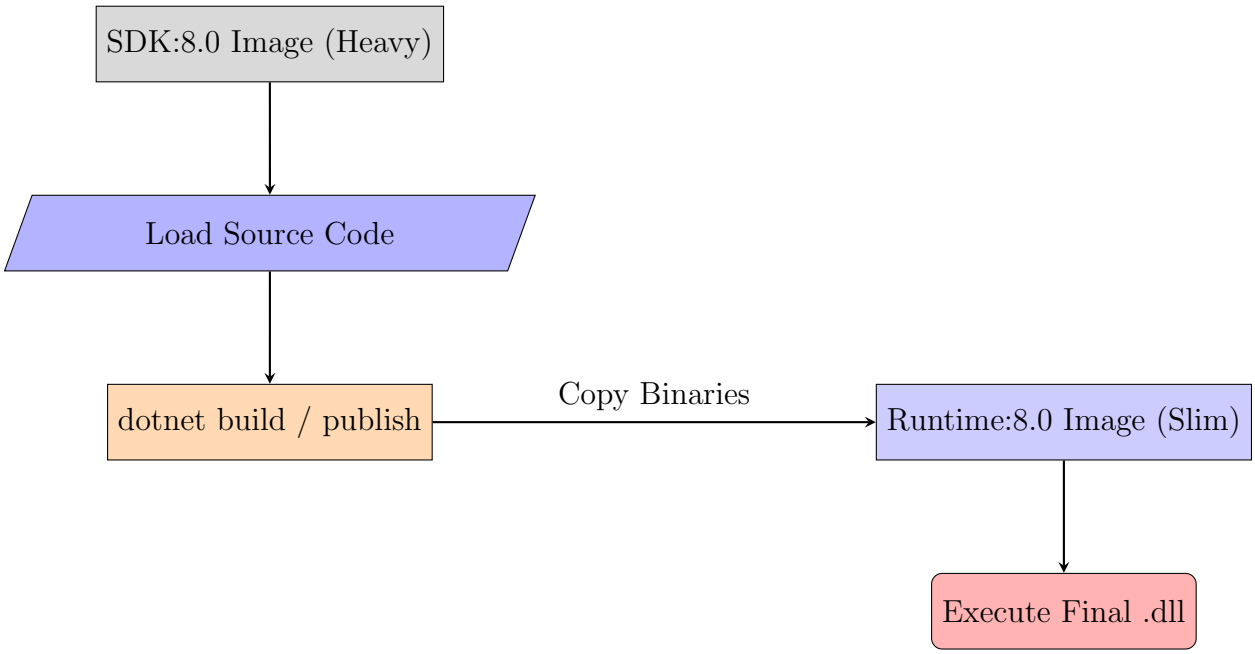

Figure 7: Multi-Stage Build Structural Shift Diagram

9.4 Run Commands

```
# Delete old container if name is blocked
docker rm my_emotion_container

# Build the Docker image
docker build -t emotion-app-image .

# Run the container
docker run --name my_emotion_container emotion-app-image
```

Listing 4: Docker Terminal Commands

10 Challenges & Solutions

Throughout the lifecycle of building and containerizing this AI-powered application, several technical hurdles were encountered. This section outlines the problems faced and the corresponding solutions applied to overcome them.

10.1 GUI Crashes in Headless Docker Environments

Problem: Windows Forms applications rely heavily on native Windows OS UI libraries (like GDI+). When deploying the application inside a Linux-based Docker runtime image, the application immediately crashed with a `TypeInitializationException` because visual handles could not be created in a headless environment.

Solution: An environmental switch was added to the application's entry point (`Program.cs`). The solution checks if the environment variable `DOTNET_RUNNING_IN_CONTAINER` is set to true. If it is, the application bypasses the standard `Application.Run(new Form1())` command and instead executes in a secure, text-based console prediction mode suitable for servers.

10.2 Dataset Delimiter Conflicts

Problem: The Kaggle dataset used natively featured comma-separated values (CSV). However, the human emotion text blocks contained standard conversational punctuation (like commas), which caused the ML.NET `LoadFromTextFile` schema to split rows prematurely, leading to corrupted column indices and training failures.

Solution: Before fitting the data, a pre-processing step was executed on the `emotions.txt` file to scrub standard commas and map the separation mapping strictly to the semicolon (`;`) delimiter. This allowed conversational text blocks to remain intact without disrupting data schema mapping.

10.3 Docker Build Context and File Pathing

Problem: During the Docker `COPY` step, the application initially failed to train inside the container because the file paths mapped natively in Windows (e.g., pointing towards the execution folder's `bin/Debug`) did not exist in the isolated Linux runtime path (`/app`).

Solution: Code paths targeted towards the training dataset and the generated `.zip` files were changed to relative paths based on `AppContext.BaseDirectory`. Additionally, the Dockerfile was mapped to copy and maintain relative text file paths directly beside the executing `.dll` binaries.

11 Conclusion

The EmotionApp project successfully demonstrates continuous local pipeline training and AI prediction utilizing ML.NET and .NET 8.0. The setup covers all expected components, from custom dataset parsing to environment-aware GUI bypasses allowing deployment in headless server environments.

References

1. Microsoft. (2024). ML.NET Documentation. https://learn.microsoft.com/en-us/dotnet/machine-learning/
2. Microsoft. (2024). Windows Forms Documentation. https://learn.microsoft.com/en-us/dotnet/desktop/winforms/
3. Docker. (2024). Docker Documentation. https://docs.docker.com/
4. Emotion Dataset for Sentiment Analysis. https://www.kaggle.com/datasets/praveengovi/emotions-dataset-for-nlp